

The Hong Kong Polytechnic University
Department of Computing

COMP4913 Capstone Project
Report (Final)

**Compress-Transfer-Decompress for LLM
Serving: cuSZp-Enabled CPU–GPU Data
Pipeline in vLLM**

Student Name:	KONG Zirui
Student ID No.:	22103493D
Programme-Stream Code:	61435-FCS
Supervisor:	Dr ZHANG Zhaorui
Co-Examiner:	Prof. WU Yujie
2 nd Assessor:	Prof. YANG Hongxia
Submission Date:	10 April 2026

Abstract

As context windows in Large Language Models (LLMs) expand, the Key-Value (KV) cache consumes massive amounts of memory during inference, which creates a severe bottleneck. When GPU VRAM runs out, serving engines like vLLM will avoid crashes by swapping inactive KV cache blocks to the host CPU. However, sending all this data back and forth across the PCIe bus will create a huge time latency, which slows down the entire inference process.

To address this bottleneck, this project introduces a “Compress-Transfer-Decompress” data pipeline integrated directly into vLLM. The approach uses cuSZp, a fast lossy compression framework that runs on the local GPU. A custom C++ wrapper uses PyBind11 to connect the underlying CUDA APIs to vLLM’s Python CacheEngine. Instead of modifying the core engine code, the Monkey-Patching technique was used to intercept the KV cache tensors. The system compresses them before they hit the PCIe bus and decompresses them when the engine needs. This greatly reduces the physical payload.

The setup was tested on an NVIDIA RTX 5080 GPU. Instead of using dummy data, real Layer-0 key embeddings were extracted from eight different models, including Qwen2.5, GPT-2, and OPT. An error bound of 1×10^{-4} was set to ensure the model generation quality remained safe. As the GPU handles compression so quickly, the time saved on PCIe transfer easily covers the computational overhead. Testing showed the effective Swap-Out bandwidth improved to $1.77\times$, and Swap-In bandwidth improved to $2.08\times$. The pipeline reduced the data from $2.42\times$ to $3.06\times$. This proves that compressing tensors at runtime is a practical method to bypass PCIe limitations in LLM services.

Acknowledgments

First of all, I would like to express my deepest gratitude to my supervisor, **Dr. ZHANG Zhaorui**, for her valuable guidance, continuous encouragement, and helpful suggestions throughout the research and development process of this project. Her expertise in systems and performance optimization provided the foundation for this work.

I am also sincerely grateful to **Prof. WU Yujie** and **Prof. YANG Hongxia** for their time in reviewing the thesis. Their professional suggestions have significantly enhanced the quality of this final report.

Special thanks to the Department of Computing at The Hong Kong Polytechnic University for providing essential research facilities and an engaging academic environment.

Personally, I want to thank my family and my beloved for their unwavering support and trust. Stepping out of my comfort zone in front-end development and AI/robotics research to study the complexities of CUDA-accelerated systems has been a challenging journey. Fortunately, having a better half who also studies computer science provided me with inspiration and comfort; her patience and encouragement were my greatest motivations.

Finally, as I prepare to start my Master's studies at The Chinese University of Hong Kong, I look back on my undergraduate years at PolyU with immense gratitude for the personal growth and opportunities I gained.

Contents

Abstract	2
Acknowledgments	3
List of Tables	6
List of Figures	7
1 Introduction	8
1.1 Motivation	8
1.2 Problem Statement	8
1.3 Project Objectives	8
1.4 Key Contributions	9
1.5 Thesis Organization	9
2 Background and Literature Review	10
2.1 LLM Serving and the KV Cache	10
2.1.1 Autoregressive Generation and the KV Cache	10
2.1.2 vLLM and PagedAttention	10
2.2 Memory Bottlenecks in GPU Computing	10
2.3 Data Compression for Tensors	11
2.3.1 Lossless and Lossy Compression	11
2.3.2 Error-Bounded Lossy Compression	11
2.4 Introduction to cuSZp	12
3 Methodology	13

3.1	System Integration Strategy	13
3.2	The Compress-Transfer-Decompress Method	14
3.3	Dynamic Error Bound Selection and Management	15
4	System Design and Implementation	16
4.1	C++ Backend and cuSZp Integration (cuszp_wrapper)	16
4.2	vLLM Interface with Monkey-Patching (compression_pipeline)	17
4.3	Integration and Infrastructure Deployment (docker)	18
5	Evaluation and Benchmarks	19
5.1	Experimental Setup	19
5.2	Benchmarking Pipeline	19
5.2.1	Methodology	19
5.2.2	Metrics	20
5.3	Performance Results Analysis	20
5.4	Sensitivity Analysis of Error Bounds	21
6	Discussion	24
6.1	Interpretation of Results	24
6.2	Implementation Challenges	24
6.3	Limitations	25
7	Conclusion and Future Work	26
7.1	Summary of Achievements	26
7.2	Final Remarks	26
7.3	Future Directions	26
	References	28

List of Tables

1	Compression Metrics and Effective Bandwidths vs. Uncompressed Baseline	20
2	Impact of Relative Error Bounds on GPT-2 KV Tensor Compression (16 MB Original Size)	21
3	Impact of Relative Error Bounds on Qwen2.5-1.5B KV Tensor Compression (16 MB Original Size)	22

List of Figures

- 1 Architectural flowchart of the **Compress-Transfer-Decompress** method. A large number of original kV tensors are compressed on the local GPU, and the minimized bit stream is transmitted with PCIe. The critical reconstruction metadata will be saved in the host RAM dictionary. 17
- 2 Comparison of Swap-Out and Swap-In Bandwidth across different LLMs. The cuSZp compression pipeline provides substantial improvements in effective data transmission rates over the uncompressed baseline for both bidirectional PCIe operations. 21
- 3 Trade-off analysis between Compression Ratio (blue solid) and Max Absolute Error (red dashed) on GPT-2 and Qwen2.5 models. The vertical dashed lines denote the optimal selection boundary at 10^{-4} to ensure high bandwidth reduction while preventing destructive accuracy degradation. 22

1 Introduction

1.1 Motivation

In recent years, Large Language Models (LLMs) have demonstrated unprecedented capabilities [1] across a wide range of Natural Language Processing (NLP) tasks. With the increasing demand for analyzing and generating longer documents, modern LLMs are designed to support the massive length of context windows. However, this capability has a serious memory challenge during the autoregressive inference. In particular, the key value (KV) cache used to store the tensor of previous tokens to avoid redundant recalculation grows rapidly with the increase of sequence length and batch size. To accommodate requests that exceed the Video Random Access Memory (VRAM) capacity of Graphics Processing Units (GPUs), advanced service engines such as vLLM [2] adopt memory management techniques, such as PagedAttention, dynamically offloading inactive KV cache blocks to the host Central Processing Unit (CPU) memory and restoring them when needed. While highly effective in preventing Out-Of-Memory (OOM) failures, this offloading mechanism exposes a critical hardware limitation, long-context generation, which must be addressed to maintain high-throughput.

1.2 Problem Statement

The main bottleneck [3] in KV cache swapping lies in the physical Peripheral Component Interconnect Express (PCIe) bus that connects the CPU and GPU. As PCIe bandwidth is several orders of magnitude lower than the High-Bandwidth Memory (HBM) inside the GPU, the transmission of a large number of uncompressed multidimensional tensor data between the GPU and the CPU memory will cause significant delays. This delay can substantially block the GPU computation pipeline and reduce the overall inference speed. Therefore, without optimizing the data transmission payload, PCIe bandwidth wall seriously limits the actual use effect of the KV cache.

1.3 Project Objectives

To ease the bottleneck of the above data transmission, this project aims to design a “Compress-Transfer-Decompress” data pipeline with vLLM. The specific objectives of this project are as follows:

- **Accelerate KV Cache Swapping:** To reduce the amount of physical data transmitted through the PCIe bus by using lossy compression with error limits before transmission, and quickly decompress the data when it is returned to the GPU.

- **Seamless Integration into vLLM:** To successfully embed this compressed pipeline into the vLLM PagedAttention architecture to ensure compatibility and stability without interrupting the core memory management function.

1.4 Key Contributions

To fulfill these objectives, this project uses **cuSZp** [4], a highly optimized, GPU-accelerated, and error-bounded lossy compression framework. The key contributions of this project are as follows:

- **Custom PyBind11 [5] Wrapper for cuSZp:** Designed and compiled a robust C++ wrapper via PyBind11, directly exposing the native cuSZp compression and decompression Application Programming Interfaces (APIs) to Python with minimal calls.
- **Implementation of Compressed Swap Logic in vLLM:** Developed an integration pipeline using Monkey-Patching [6] technique to intercept and override vLLM’s native `swap_in` and `swap_out` tensor block operations with Compress-Transfer-Decompress mechanism.
- **Comprehensive Automated Benchmark Suite:** Created an automatic analysis pipeline that slices Layer-0 [7] key embeddings to evaluate the system’s performance. The solution has been basically verified in eight most advanced LLMs, including GPT-2 [8], Qwen2.5 [9], OPT [10], Pythia [11], and TinyLlama [12].

1.5 Thesis Organization

The remainder of this thesis is structured as follows. **Section 2** reviews the background of LLM serving, memory bottlenecks, and the principles of error-bounded lossy tensor compression. **Section 3** introduces the methodology, including the latency trade-off model and the selection of the error bound. **Section 4** presents the system design and implementation details of the C++ wrapper and vLLM integration. **Section 5** outlines the experimental setup and evaluations of the benchmark results. **Section 6** discusses the interpretation of results, hardware utilization, and implementation challenges. **Section 7** summarizes the thesis and suggests directions for future work.

2 Background and Literature Review

2.1 LLM Serving and the KV Cache

2.1.1 Autoregressive Generation and the KV Cache

LLMs based on transformer architecture generally generate text in an autoregressive manner. This means that predicting the next token requires the model to process all preceding tokens in the sequence. During inference, especially the decoding stage, calculating the attention of a new token requires the K and V tensors of all previously generated tokens. Recomputing these tensors at every generation step is infeasible. To optimize it, the inference engine uses a mechanism called KV caching. By storing the K and V tensors in memory after the first computation, the model can reuse them in subsequent steps to achieve significant computational acceleration at the cost of memory capacity. However, as the length of the context windows and batch sizes increase, the occupied space of the KV cache grows rapidly, and it quickly becomes a main memory consumer during inference.

2.1.2 vLLM and PagedAttention

To manage the massive memory requirements of the KV Cache, **vLLM** introduced **PagedAttention**, which will partition the continuous KV Cache into fixed-size and non-contiguous blocks. When the GPU VRAM is exhausted due to incoming requests or long sequence generation, vLLM's **CacheEngine** automatically intervenes. It frees up VRAM by preempting lower-priority requests and offloading (swapping out) their associated KV blocks to the host CPU RAM. Once the GPU has sufficient resources to resume the preempted request, the **CacheEngine** restores (swaps in) the blocks to VRAM. While this swapping mechanism prevents OOM errors, it introduces severe latency overhead that depends on the hardware's data transmission rate.

2.2 Memory Bottlenecks in GPU Computing

In modern heterogeneous computing systems, the performance of data-intensive operations largely depends on memory bandwidth. There is a significant difference between the internal bandwidth of the GPU and the interconnect bandwidth that connects the GPU to the host CPU:

- **GPU VRAM Bandwidth:** High-end GPUs are equipped with High Bandwidth Memory (HBM) or ultra-fast GDDR memory (e.g., GDDR7 on the RTX 5080), capable of internal data transmission rates exceeding 1,000 GB/s.

- **CPU-GPU PCIe Bandwidth:** In contrast, data transmission between the host CPU memory and GPU VRAM must traverse the Peripheral Component Interconnect Express (PCIe) bus. Even with a modern PCIe Gen 4.0 or Gen 5.0 connection, the theoretical bandwidth is limited between 32 GB/s and 64 GB/s, and the actual effective bandwidth is usually much lower.

During the loading and offloading of KV caches in vLLM, uncompressed Gigabyte tensor data will pass through this narrow PCIe channel. These bandwidth limitations create a “Memory Wall” [13] where the GPU is waiting for data from the CPU, severely limiting the overall inference speed of LLMs.

2.3 Data Compression for Tensors

To alleviate the PCIe bandwidth bottleneck, data compression can be applied to reduce the data payload before transmission. Compression technologies are usually divided into two categories.

2.3.1 Lossless and Lossy Compression

Lossless Compression algorithms [14] guarantee bit-by-bit reconstruction of the original data. However, the floating-point tensors generated by neural networks are full of random noise in their final decimal places. As these trailing digits fluctuate constantly, the lossless compression algorithms cannot find the repeating patterns, resulting in low compression ratios (often under $1.5\times$). As a result, lossless methods cannot shrink the data enough to speed up the PCIe transmission.

Lossy Compression [15] achieves a significantly higher compression ratio by discarding less important information. Although this may introduce noise or precision loss, the neural network is inherently robust to small perturbations in the activation process. The challenge of traditional lossy algorithms is that unpredictable data loss will slowly affect the results through each layer of LLM, resulting in a significant decline in output quality (e.g., hallucination).

2.3.2 Error-Bounded Lossy Compression

To balance high compression ratios and high quality, Error-Bounded Lossy Compression frameworks have been developed. These algorithms allow developers to define mathematical error boundaries, guaranteeing that the difference between the raw data point (x_i) and the decompressed data point (x'_i) will never exceed the predefined threshold ϵ (i.e., $|x_i - x'_i| \leq \epsilon$). This high-precision control ensures that the accuracy of the KV cache is kept within a safe range while reducing the incoming tensor data.

2.4 Introduction to cuSZp

In this project, the **cuSZp** framework was used as the underlying compression engine. cuSZp is a fast lossy compression program with error bounds, which is specially designed for floating-point data. There are several reasons for using it:

- **GPU-Accelerated Parallel Computation:** Unlike CPU-based compressors, cuSZp is implemented in Compute Unified Device Architecture (CUDA), allowing it to use thousands of GPU cores. This parallel operation ensures that the time spent compressing data will not exceed the time saved during PCIe transmission.
- **High Throughput:** cuSZp focuses on high-speed encoding and decoding rather than emphasizing maximal compression ratios. This concept can directly reduce the data transmission delay in LLM calculations.
- **Precise Error Control:** cuSZp will strictly follow the user-defined absolute or relative error limits to ensure the integrity of the KV cache and maintain the generation quality of LLMs.

3 Methodology

This section introduces the details of the system architecture and implementation strategy of integrating the cuSZp lossy compression program into the vLLM inference engine. This integration aims to alleviate the PCIe bandwidth bottleneck while maintaining structural compatibility with the existing memory management of vLLM

3.1 System Integration Strategy

This integration is implemented without modifying the vLLM source code. Instead, a Monkey-Patching technique was used to intercept `CacheEngine` mechanisms of vLLM. Specifically, the default methods `_swap_out_blocks_to_host` and `_swap_in_blocks_from_host` in the `vllm.worker.cache_engine` module are replaced with the new functions that handle the compression logic at runtime via a special `CompressedCacheEngineMonkeyPatch` class. The underlying C++ cuSZp API was linked directly to Python using PyBind11. To standardize the data, the `CuSZpCompressor` module transforms all incoming tensors into flattened 1-Dimension (1-D) arrays of single-precision floats.

Another critical challenge addressed in this integration is synchronization. Typically, using a custom CUDA kernel within PyTorch [16] will cause a pause to wait for synchronization. When the compression program runs on different channels, the GPU needs frequent pauses to synchronize data streams, resulting in serious pipeline congestion. To solve this, the wrapper of the program will directly extract the current PyTorch active CUDA stream with the `c10::cuda::getCurrentCUDASTream().stream()` API. All compression and decompression tasks were dispatched to this shared stream. This low-level trick completely avoids synchronization pauses. The customized compressor can queue the workload perfectly and synchronize with the PyTorch asynchronous operation of vLLM.

By the original design, vLLM’s native `BlockSpaceManager` only handles fixed-size memory blocks. Since compressed data streams are variable in length, relying on the standard CPU cache allocation is not feasible. To work around this limitation, the system implements a separate Python dictionary named `compressed_blocks_store`. This dictionary is used as a custom tracking system. It indexes every compressed block on the CPU using a composite key, which means that the layer index is combined with the destination block index `((layer_idx, dst_idx))`. Alongside the byte stream, the store also saves the metadata required for later decompression, specifically the original tensor size, structural shape, and the calculated absolute error bound (`actual_eb`).

Additionally, a strict safety measure has been formulated. If a tensor block resists any compressions due to compression error, the system simply cancels the compression attempt. It then immediately

falls back to the original vLLM routine (`_swap_out_blocks_to_host_original()`) to transfer the original and uncompressed payload safely.

3.2 The Compress-Transfer-Decompress Method

By overriding vLLM’s memory operations, the system changes from a direct physical transfer model to a *Compress-Transfer-Decompress* method to maximize PCIe throughput efficiency:

Algorithm 1 Dynamic Error-Bounded Compressed Swap-Out Pipeline

Require: Uncompressed KV tensor T_{gpu} , Relative Error Bound ϵ_{rel} , Target CPU indices (L_{idx}, D_{idx})

Ensure: Compressed bitstream B_{cpu} stored in Host RAM with metadata

```

1:  $R \leftarrow \max(T_{gpu}) - \min(T_{gpu})$  {Calculate range via CUDA reduction}
2: if  $R < 10^{-6}$  then
3:    $R \leftarrow 10^{-6}$  {Prevent the precision collapse of tensors}
4: end if
5:  $\epsilon_{abs} \leftarrow \epsilon_{rel} \times R$  {Compute the absolute error bound}
6:  $S_{est} \leftarrow \text{EstimateBufferSize}(\text{sizeof}(T_{gpu}))$ 
7:  $B_{gpu} \leftarrow \text{Allocate}(\text{uint8}, S_{est}, \text{device}='cuda')$ 
8:  $(Status, S_{actual}) \leftarrow \text{cuSZp\_compress}(T_{gpu}, B_{gpu}, \epsilon_{abs})$ 
9: if  $Status \neq \text{SUCCESS}$  then
10:  return OriginalUncompressedTransfer( $T_{gpu}$ )
11: end if
12:  $B_{minimized} \leftarrow B_{gpu}[0 : S_{actual}]$  {Slice to exact compressed byte length}
13:  $B_{cpu} \leftarrow \text{AsyncTransferToHost}(B_{minimized})$ 
14:  $\text{MetadataStore}[(L_{idx}, D_{idx})] \leftarrow \{B_{cpu}, \text{shape}(T_{gpu}), \epsilon_{abs}\}$ 
15: return SUCCESS

```

- **Swap-Out (GPU to CPU):** When a KV block is swapped out by the cache manager (vLLM), the system compresses it directly on the GPU using `cuSZp`. Since the final size of the compressed byte stream is unpredictable, the C++ wrapper allocates an oversized, uninitialized GPU buffer (up to $2\times$ the original tensor size) using `torch.empty` to prevent stream overflow first. After the compression kernel runs, the output usually takes up only a small fraction of this allocated space. The code then slices the large buffer down to its exact byte length using `[:actual_size]`. Only this precise, minimized byte array crosses the PCIe bus to the host CPU via a `.cpu()` call. This step ensures no empty buffer space is transmitted, effectively bypassing the transfer latency bottleneck. Finally, the system saves the compressed bitstream and its metadata into the `compressed_blocks_store` dictionary, and updates internal counters to track the real-time compression ratio (as outlined in Algorithm 1).
- **Swap-In (CPU to GPU):** When a swapped block is needed again, the system queries the host dictionary using the block index to fetch both the compressed bytes and the stored metadata. The code then pushes this byte stream back to the GPU. It specifically uses PyTorch’s

`tensor.cuda(non_blocking=True)` to handle this transfer asynchronously, ensuring the main Python process never pauses while waiting for memory operations.

Once the data reaches the GPU, the `cuSZp_decompress` kernel will perform the remaining operations. It will read the exact `actual_eb` value saved earlier to reverse the compression. Finally, the kernel will write the restored KV cache directly into vLLM’s pre-allocated target buffer (`_dst_cache`), making it available for the next inference step.

3.3 Dynamic Error Bound Selection and Management

The `cuSZp` compressor only accepts an absolute error bound. However, the range of KV cache activations fluctuates wildly depending on the layer or token. Applying a static absolute error will either destroy data accuracy or fail to compress effectively. To solve this, the pipeline dynamically converts a relative error bound (ϵ_{rel}) into an absolute one before compression.

The C++ code first uses **CUDA reduction primitive** to find the maximum and minimum values of the target tensor block. It calculates the data range as $R = \max(\mathbf{T}) - \min(\mathbf{T})$. If a tensor contains mostly identical numbers ($R \approx 0$), the code enforces a strict minimum threshold of 1×10^{-6} to prevent mathematical precision collapse. The system then calculates the final absolute error bound (ϵ_{abs}) using the following formula:

$$\epsilon_{abs} = \epsilon_{rel} \times \max(R, 10^{-6}) \quad (1)$$

The decompression kernel requires this exact ϵ_{abs} value to rebuild the tensor. Therefore, the pipeline extracts this generated `actual_eb` from the C++ output and stores it safely in the Python `compressed_blocks_store`. During a swap-in request, the system injects this saved scalar back into the decompressor, ensuring the KV cache block is reconstructed accurately.

4 System Design and Implementation

This section provides a detailed explanation of the system architecture and specific implementation of the cuSZp compression pipeline in the vLLM framework. The design connects underlying C++ and CUDA acceleration with an integrated Python framework via PyBind11, eventually forming a container deployment environment to ensure repeatable execution across different hardware systems.

4.1 C++ Backend and cuSZp Integration (`cuszp_wrapper`)

The core compression is encapsulated in an optimized C++ wrapper located in the `integration/cuszp/_wrapper` directory. This module directly cooperates with the native cuSZp library and PyTorch’s C++ API (LibTorch) to perform operations entirely on the GPU, avoiding costly synchronization between the host and device during the computation process.

- **`cuszp_wrapper.cpp`, `cuszp_wrapper.h`:** These files define the `CuSZpWrapper` class, which manages the initialization, memory lifecycle, and kernel execution of the cuSZp framework. To support varying tensor magnitudes across different LLM layers, the wrapper implements dynamic error-bound calculation. It implicitly uses CUDA reduction primitives via PyTorch’s native `input_tensor.min().item<float>()` and `input_tensor.max().item<float>()` functions to mathematically locate the true numerical boundaries of the incoming tensor block. These bounds are subsequently used to convert user-defined relative error bounds into strictly enforced absolute errors for the cuSZp kernel. Both the `compress` and `decompress` methods accept PyTorch tensors (`torch::Tensor`), strictly enforcing via `.is_cuda()` checks that the underlying memory pointers reside entirely on the GPU device to maximize end-to-end throughput. If the dynamically supplied output buffer is insufficient to hold the output bitstream, the wrapper automatically reallocates a new PyTorch memory block, explicitly scaled to double the input tensor’s memory footprint (`estimated_size = original_size * 2`).
- **`pybind11_bindings.cpp`:** To expose the compiled C++ objects securely and efficiently to the Python interpreter, PyBind11 is employed. This binding file defines the `cuszp_wrapper_cpp` Python module. It maps necessary configuration structures, such as `CompressionConfig`, to fully accessible Python classes. Importantly, to ensure safe asynchronous execution in vLLM, the wrapper maintains the current active PyTorch CUDA stream, passing it through to the cuSZp library calls. After launching the cuSZp API, the C++ code will call `cudaStreamSynchronize(stream)`. This will create a hard stop, forcing the system to wait until the GPU completely finishes the compression task. Without this strict synchronization, vLLM might attempt to read or overwrite the memory buffer before compression is complete, which will cause data race conditions.

- **CMakeLists.txt:** CMake handles the entire C++17 build process. This configuration script, as an automatic program, locates the NVCC compiler, Python3 headers, PyBind11, and LibTorch dependencies without manual path setting. During the compilation stage, the script links the project objects with `libcusZp.a` and `libtorch_python.so`. The final output is a single dynamic shared library (`cuszp_wrapper_cpp.so`). This compiled file allows the Python environment to import the C++ wrapper like importing standard Python modules.

4.2 vLLM Interface with Monkey-Patching (`compression_pipeline`)

Integrating the cuSZp compressor into vLLM requires efficient methods to avoid modifying the original source code of the vLLM engine. To achieve it, the system needs to use Monkey-Patching during the Python runtime. This technique intercepts and replaces the default memory routines while the program is running, leaving the kernel installation unchanged. Figure 1 draws the complete architectural data flow managed by the custom Python interface.

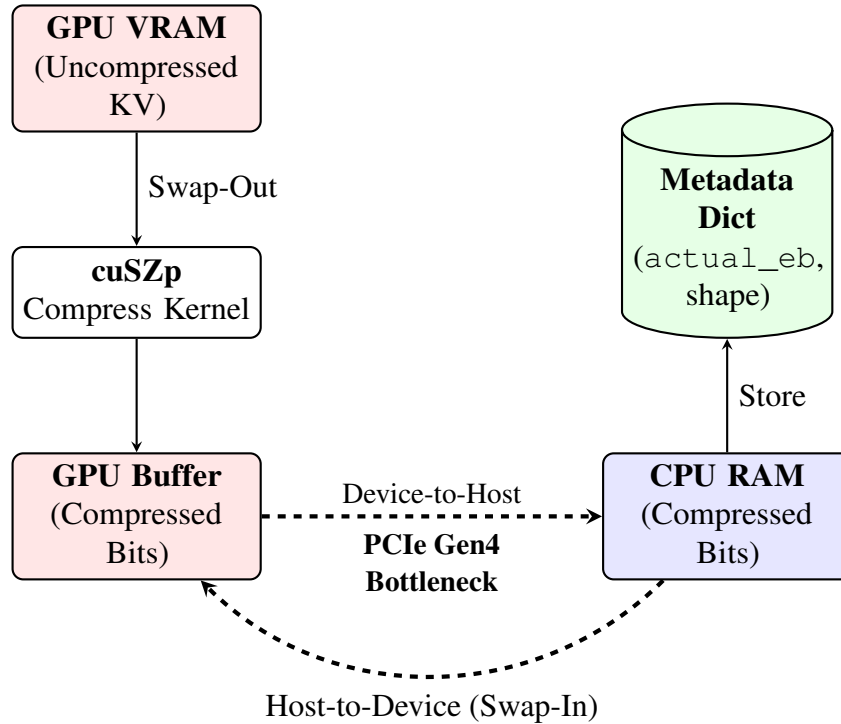


Figure 1: Architectural flowchart of the **Compress-Transfer-Decompress** method. A large number of original kV tensors are compressed on the local GPU, and the minimized bit stream is transmitted with PCIe. The critical reconstruction metadata will be saved in the host RAM dictionary.

- **compressed_swap.py:** This script connects vLLM to the compiled C++ backend. It defines a `CuSZpCompressor` class to configure the C++ wrapper with necessary error limits, `DIM_1D` formatting, and GPU device mapping. The script also contains the `CompressedCacheEngineMonkeyPatch` class. When vLLM initializes, this patch

class will automatically override the engine's default `_swap_out_blocks_to_host` and `_swap_in_blocks_from_host` methods.

For swap-out operations, the new function passes the raw GPU cache block to the compressor. The code then slices the resulting byte stream to remove the unused buffer and sends the exact payload to the CPU only, which significantly reduces transmission time. It records the compressed data and necessary reconstruction metadata, such as layer indices, tensor shape, and the specific `actual_eb`, directly into the `compressed_blocks_store` dictionary.

For swap-in operations, the system retrieves the exact byte stream using its dictionary key and pushes it back to the GPU asynchronously via `tensor.cuda(non_blocking=True)`. Finally, the decompressor reconstructs the data to exactly restore the vLLM cache block.

- **test_vllm_integration.py:** This script serves as a standalone unit test to verify the Monkey-Patching mechanism before running actual language models. The code sets up a `MockCacheEngine` to simulate vLLM's internal KV cache structure. It fills this mock cache with pseudo-random numbers and enforces the swap cycle. This test not only verifies that the Python hooks successfully intercept the memory routines but also confirms that the decompressed tensors match the original tensors within the allowable error bound.

4.3 Integration and Infrastructure Deployment (docker)

Managing dependencies like specific PyTorch versions, the CUDA toolkit, and the CMake compiler across different platforms (such as Linux and Windows WSL2) often leads to environment conflicts. To ensure multi-platform stability, the system isolates the entire execution environment in the Docker container.

- **Dockerfile:** The build starts from the `nvidia/cuda:12.1.1-devel-ubuntu22.04` base image to secure access to the NVCC compiler. The script will then install Python 3.10, essential system tools, and required pip packages. Instead of relying on pre-compiled binary files, the Dockerfile pulls the `cuSZp` source code directly from GitHub and compiles it globally. It then exports the required `'LD_LIBRARY_PATH'` so the custom `PyBind11` wrapper can locate the CUDA libraries at runtime without triggering linking errors.
- **run.sh:** Manually typing a long Docker command is cumbersome and error-prone, so a dedicated shell script will manage the container lifecycle. It automatically fixes common Windows-to-Linux line-ending bugs (converting CRLF to LF) before execution. The script provides simple arguments to construct the image (`build`), open an interactive shell with GPU access (`run`), or execute the full benchmark script (`test`). This setup keeps the host machine completely clean while allowing users to trigger the entire pipeline using a single terminal command.

5 Evaluation and Benchmarks

5.1 Experimental Setup

The hardware baseline for these tests is a personal workstation equipped with an NVIDIA RTX 5080 GPU. This specific computation card utilizes high-bandwidth PCIe lanes, establishing a solid foundation for standard CPU-GPU data transmission speeds.

The software environment runs on Linux and relies on PyTorch, vLLM, and the HuggingFace Transformers [17] library. To prevent library version conflicts and ensure the tests are reproducible on other machines, Docker and the NVIDIA Container Toolkit have isolated the entire software stack, including the custom PyBind11 wrapper and CMake build tools.

5.2 Benchmarking Pipeline

Using random floating-point noise to test memory bandwidth can not accurately reflect the behavior of LLMs in production. Therefore, a custom Python script (`benchmark_pipeline.py`) was built to capture and test true LLM cache states.

5.2.1 Methodology

The pipeline script loads eight distinct advanced LLMs. To force these models to generate realistic, long-context hidden states, the tokenizer repeatedly ($30\times$) injects a dense informational prompt “*The Hong Kong Polytechnic University (PolyU) is a public research university located in Hung Hom, Hong Kong.*”.

During the text generation stage, the script intercepts the actual Layer-0 key embeddings (`past\_key\_values[0][0]`). It flattens these multi-dimensional arrays and reshapes them to 4,194,304 elements (approximately 16 MB in FP32). This step normalizes the memory footprint across all models while completely preserving the numerical distribution and local extreme values of tokens.

To record accurate timestamps without being affected by asynchronous GPU scheduling delays, the script places `torch.cuda.synchronize()` barriers around the host clock (`time.perf\_counter()`). Before recording any data, the system runs a mandatory warm-up stage: 10 dummy transfers for the uncompressed baseline, and 5 preliminary compression cycles to allocate all necessary VRAM pages. The script then executes 50 active iterations and averages the results to determine the final compression time, decompression latency, and effective bandwidth.

5.2.2 Metrics

The automated script tracks three main metrics:

- **Compression Ratio:** The original tensor byte size divided by the size of the final compressed byte stream.
- **Absolute Max Error:** The mathematical difference between the original numbers and the decompressed numbers, ensuring it stays within the target bound of $\epsilon = 1 \times 10^{-4}$.
- **Baseline vs. Effective Bandwidth:** A comparison between the physical PCIe transfer speed (uncompressed) and the new effective transfer speed, which considers both the reduced payload size and the GPU compression/decompression time cost.

5.3 Performance Results Analysis

The script exports all execution logs into JSON files and compiles them as a Markdown summary. As Table 1 and Figure 2 indicate, applying lossy compression to real KV cache tensors can obtain a performance gain.

In the architecture of the entire test, the pipeline reduces the physical data payload by $2.42\times$ to $3.06\times$. The maximum absolute error remains safely around the 10^{-3} . By significantly reducing the size of the physical payload, the time saved on the PCIe bus can exceed the millisecond overhead of executing the cuSZp core on the GPU.

The Qwen2.5-1.5B model showed the highest single gain, hitting a $2.08\times$ speedup for Swap-In operations. This effectively doubled the bandwidth from 16.32 GB/s to 33.86 GB/s. Other architectures like GPT-2, OPT, and Pythia have achieved extremely high speedup, which proves that the memory wall can be reliably bypassed by using the structural redundancy in the KV cache.

Table 1: Compression Metrics and Effective Bandwidths vs. Uncompressed Baseline

Model	Comp. Ratio	Max Error	Base Swap-Out	Effective Swap-Out	Out Speedup	Base Swap-In	Effective Swap-In	In Speedup
GPT-2 (124M)	$2.69\times$	2.06×10^{-3}	8.25 GB/s	14.60 GB/s	1.77 $\times$	17.16 GB/s	32.48 GB/s	1.89 $\times$
Qwen2.5-0.5B	$2.42\times$	2.52×10^{-2}	14.29 GB/s	19.53 GB/s	1.37 $\times$	17.92 GB/s	30.52 GB/s	1.70 $\times$
Qwen2.5-1.5B	$3.06\times$	6.08×10^{-2}	14.56 GB/s	22.24 GB/s	1.53 $\times$	16.32 GB/s	33.86 GB/s	2.08 $\times$
OPT-125M	$2.63\times$	1.27×10^{-3}	15.05 GB/s	21.61 GB/s	1.44 $\times$	20.75 GB/s	37.36 GB/s	1.80 $\times$
OPT-350M	$2.68\times$	3.17×10^{-4}	12.33 GB/s	18.60 GB/s	1.51 $\times$	16.52 GB/s	31.00 GB/s	1.88 $\times$
Pythia-160M	$2.69\times$	2.83×10^{-3}	13.20 GB/s	19.77 GB/s	1.50 $\times$	16.63 GB/s	32.25 GB/s	1.94 $\times$
Pythia-410M	$2.79\times$	2.56×10^{-3}	14.41 GB/s	21.78 GB/s	1.51 $\times$	18.38 GB/s	35.41 GB/s	1.93 $\times$
TinyLlama-1.1B	$2.85\times$	2.04×10^{-3}	14.29 GB/s	21.69 GB/s	1.52 $\times$	18.93 GB/s	36.11 GB/s	1.91 $\times$

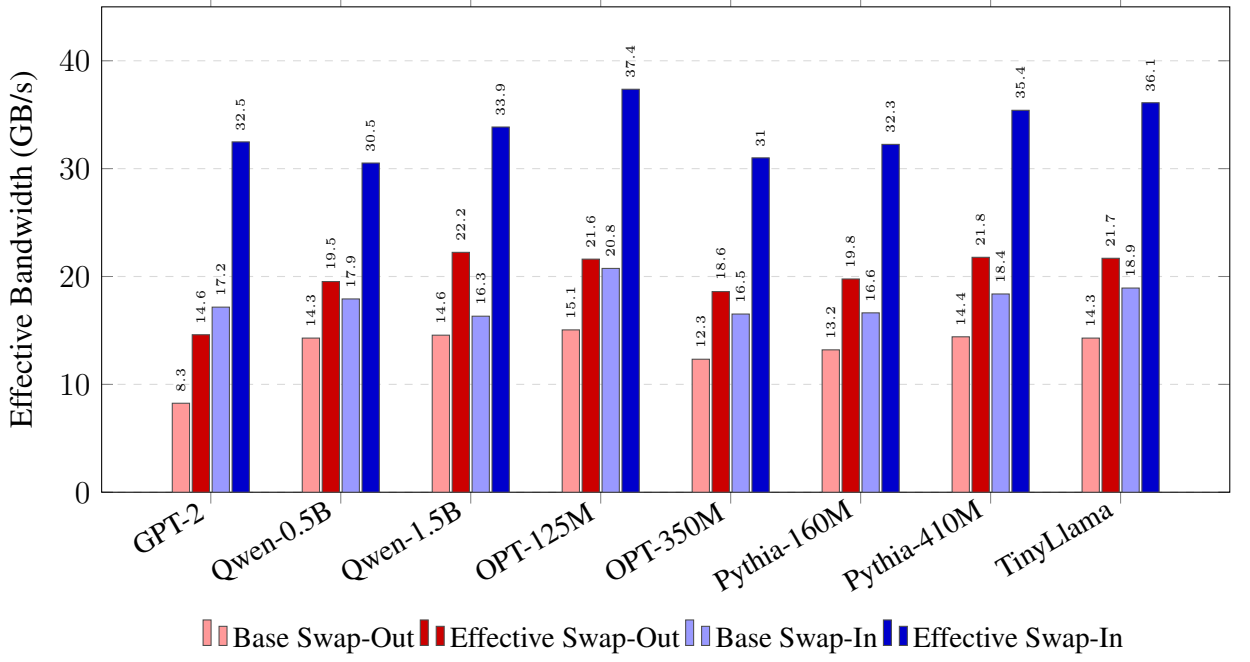


Figure 2: Comparison of Swap-Out and Swap-In Bandwidth across different LLMs. The cuSZp compression pipeline provides substantial improvements in effective data transmission rates over the uncompressed baseline for both bidirectional PCIe operations.

5.4 Sensitivity Analysis of Error Bounds

To map the exact trade-offs between tensor error and hardware speed, the script ran a sensitivity test on the GPT-2 and Qwen2.5 models. The relative error bound (ϵ_{rel}) was incrementally moved from 10^{-2} to 10^{-5} . The resulting latencies and ratios are recorded in Table 2 and Table 3.

Table 2: Impact of Relative Error Bounds on GPT-2 KV Tensor Compression (16 MB Original Size)

Rel. Error Bound (ϵ_{rel})	Comp. Ratio	Max Abs Error	Comp. Time (ms)	Decomp. Time (ms)
1×10^{-2}	$6.02\times$	2.06×10^{-1}	0.74	0.47
1×10^{-3}	$3.75\times$	2.06×10^{-2}	0.78	0.51
1×10^{-4} (Def.)	$2.69\times$	2.06×10^{-3}	0.76	0.50
1×10^{-5}	$2.10\times$	2.06×10^{-4}	0.81	0.64

As documented in Table 2, Table 3, and visualized in Figure 3, relaxing the error bound to 10^{-2} creates large compression ratios ($6.02\times$ for GPT-2 and $8.25\times$ for Qwen2.5), significantly reducing the physical PCIe payload. However, this is at the cost of data corruption. At this loose boundary, the absolute error of Qwen2.5 is more than 6.0, and the error of GPT-2 reaches 0.2. In the context of transformer attention mechanisms, exponential Softmax operations seriously magnify these raw numerical deviations and completely destroy the quality of autoregressive generation of the model. On the contrary, forcing a tight bound of 10^{-5} can make the tensor almost the same as the original tensor, but the compression ratio will be reduced to about $2.10\times$.

In addition, the logs always show that the execution time of the cuSZp compression kernel remains

Table 3: Impact of Relative Error Bounds on Qwen2.5-1.5B KV Tensor Compression (16 MB Original Size)

Rel. Error Bound (ϵ_{rel})	Comp. Ratio	Max Abs Error	Comp. Time (ms)	Decomp. Time (ms)
1×10^{-2}	$8.25\times$	6.07×10^0	0.77	0.47
1×10^{-3}	$4.41\times$	6.07×10^{-1}	0.87	0.60
1×10^{-4} (Def.)	$3.06\times$	6.08×10^{-2}	0.81	0.55
1×10^{-5}	$2.30\times$	6.08×10^{-3}	0.82	0.58

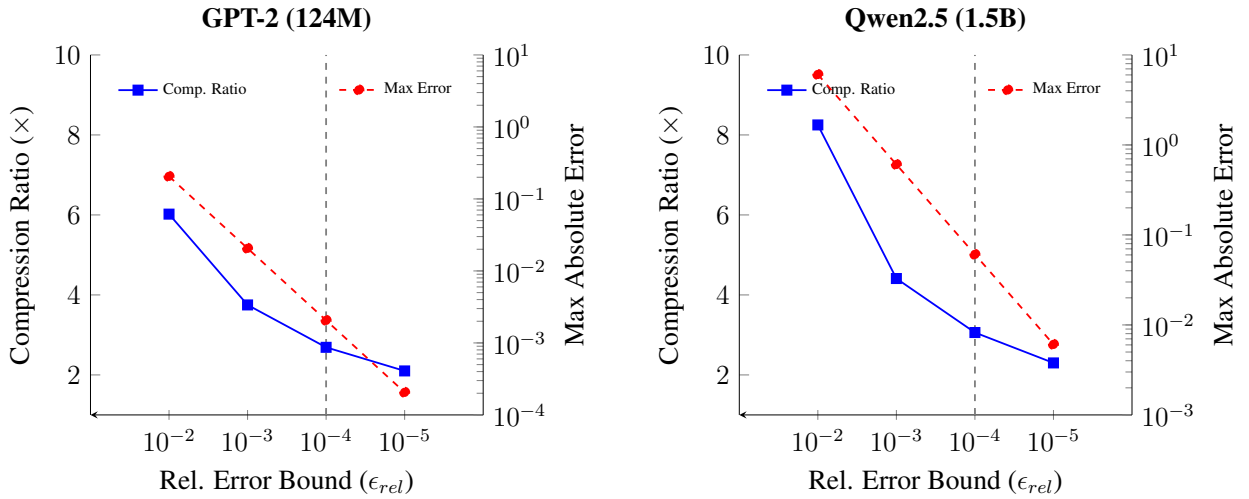


Figure 3: Trade-off analysis between Compression Ratio (blue solid) and Max Absolute Error (red dashed) on GPT-2 and Qwen2.5 models. The vertical dashed lines denote the optimal selection boundary at 10^{-4} to ensure high bandwidth reduction while preventing destructive accuracy degradation.

relatively stable (around 0.76 ms to 0.82 ms), regardless of how strict the error configuration is. This proves that on the RTX 5080, the bottleneck of the algorithm is simply the physical reading and writing to GPU VRAM, rather than the complexity of the internal mathematical branches. Based on these observations, 1×10^{-4} is the optimal relative error bound across different model families. It can greatly reduce the payload size and provide effective bandwidth acceleration without damaging the internal state of the neural network.

6 Discussion

6.1 Interpretation of Results

Benchmark logs reveal a significant gap in compression ratios between different architectures. For example, Qwen2.5-1.5B reached a $3.06\times$ ratio, while 0.5B with a small parameter reached a peak of $2.42\times$. This change can be attributed to the original data of the Key-Value (KV) cache tensors generated by each particular attention head. Naturally generated tensor models with a narrow range of values, high sparsity, or repetitive patterns will only give the cuSZp algorithm more space to compress data without breaking the strict absolute error limit.

The hardware configuration also proves that the NVIDIA RTX 5080 can reduce the algorithmic cost of the compression kernels. Powered by high-speed GDDR7 memory and modern multiprocessors, the GPU can execute the cuSZp routines in less than one millisecond. Due to the fast computation, the time saved by transmitting smaller payloads on the PCIe bus completely offsets the computational cost of compression. This raw hardware throughput is the main reason why the pipeline successfully provides end-to-end acceleration, such as the $2.08\times$ improvement recorded during the tests.

6.2 Implementation Challenges

Building and deploying this compression pipeline exposed several low-level technical barriers, mainly involving PyTorch memory safety, parallel execution timing, and environment building.

- **Memory Contiguousness:** Transferring data directly from Python to C++ requires strict memory layouts. After vLLM slicing or reshaping matrices for PagedAttention, the underlying PyTorch tensors are usually no longer stored in continuous memory blocks. If the C++ backend tries to read a fragmented tensor using the raw memory pointer, it will get data from the wrong physical address, which will immediately lead to data corruption and segmentation faults. To prevent this, the code calls `.contiguous()` on each target tensor, forcing the use of a unified memory layout before passing any pointers to the cuSZp kernels.
- **CUDA Stream Synchronization:** Managing PCIe transfers with active GPU computations is highly vulnerable to competitive conditions. Early iterations showed the CPU attempts to read memory before an asynchronous transfer finished, or start the decompressor before the byte stream completely reaches VRAM. To resolve this timing clash, the C++ wrapper pulls the active stream via `c10::cuda::getCurrentCUDAStream()`. This will force the custom cuSZp kernels to queue up safely in the native PyTorch execution process of vLLM, locking the operation order without pausing the GPU.

- **Cross-Platform Compilation:** Compiling the C++ codebase across different operating systems caused unexpected build failures. Moving project files between Windows (WSL2) and native Linux systems introduced hidden CRLF versus LF line-ending conflicts. These invisible returns completely broke the shell execution inside the Docker `run.sh` scripts and crashed the CMake configuration. The final deployment architecture includes automatic `sed` commands to clear these files before compilation, ensuring the container builds correctly regardless of the host machine.

6.3 Limitations

Although the transmission performance has been improved, the compression offload framework still has some noteworthy limitations:

- **VRAM Overhead for Workspace Buffers:** A significant disadvantage is that running the cuSZp algorithm requires additional GPU VRAM. To prevent memory overflows, the compressor forces the system to allocate temporary workspace arrays and an oversized destination buffer for the unpredictable byte stream. Although the code frees this memory almost immediately after compression, the sudden spike in VRAM usage steals space from vLLM’s active request pool. This creates a slight engineering paradox: the system must spend precious GPU memory momentarily just to execute the memory-saving offload operation.
- **Diminishing Returns on Small Batch Sizes:** The efficiency of the compressed pipeline is sensitive to the workload. In cases involving small batch sizes or extremely short context lengths, the absolute amount of KV cache being swapped is very small. In such cases, the fixed algorithm overhead caused by launching the CUDA compression kernel, combined with the baseline latency of initiating PCIe transfers, may completely offset the time savings achieved by transferring less data. Therefore, this framework is most advantageous in memory-bound, high-throughput environments (e.g., massive concurrent serving or processing extremely long documents) rather than highly latency-sensitive, single-batch inference tasks.

7 Conclusion and Future Work

7.1 Summary of Achievements

This Final Year Project designed, implemented, and evaluated a proof-of-concept data pipeline, demonstrating that enabling cuSZp compression can effectively accelerate key-value (KV) cache swapping operations in the vLLM inference engine. By utilizing a high-performance PyBind11 wrapper and dynamic Python Monkey-Patching, the proposed system transparently intercepts vLLM’s standard memory manager. It applies hardware-accelerated, error-bounded lossy compression to drastically reduce the physical data footprint transferred between GPU VRAM and CPU host memory. Large-scale automated benchmarking on eight of the most advanced LLM architectures confirmed that the computational overhead introduced by the cuSZp algorithm is more than offset by the reduction in PCIe transfer time. Finally, this pipeline produces effective bandwidth acceleration (up to $2.08\times$), which can strictly maintain the quality of generation while alleviating the hardware bottleneck.

7.2 Final Remarks

The benchmark results from this project confirm that error-bounded lossy compression is a highly practical memory optimization for modern LLM serving. As context windows grow larger, simply buying more VRAM or upgrading to faster PCIe channels is rapidly becoming too expensive for most deployments. Splicing a high-speed software compressor like cuSZp directly into the KV cache lifecycle offers a direct workaround for the hardware memory wall. This approach proves that large-scale LLMs [18] can run efficiently without a lot of hardware expansion.

7.3 Future Directions

Although the current monkey-patched pipeline successfully accelerates cache offloading, future development can push the performance limits through three specific optimizations:

- **Adaptive Error Bound Tuning:** The system currently calculates absolute error bounds based on a fixed relative percentage. Future versions can analyze attention weights [19] or token semantics during runtime to determine how sensitive a specific layer is. Then the pipeline can apply active compression to robust layers while strictly protecting fragile layers, pushing the compression ratio higher without breaking the text generation logic.
- **Native vLLM Core Integration:** Using Python Monkey-Patching and PyBind11 bridges introduces a slight context-switching delay. Rewriting the compression logic directly into

the official C++ code base of vLLM will eliminate this situation. Injecting cuSZp directly into the engine's `BlockSpaceManager` and native `PagedAttention` kernels will allow the system to allocate and track variable-length memory blocks natively.

- **Asynchronous Chunk Pipelining:** Memory scheduling may become more aggressive in the next step. Instead of processing an entire KV cache block at once, the system can split the blocks into smaller chunks [20]. This will enable a pipeline: while the CPU transfers chunk N over the PCIe bus, the GPU compresses chunk $N + 1$ at the same time. Running these operations synchronously will hide the compression delay and keep the hardware fully saturated.

References

- [1] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, “A comprehensive overview of large language models,” *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 5, Aug. 2025. [Online]. Available: <https://doi.org/10.1145/3744746>
- [2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [3] A. Li, S. L. Song, J. Chen, J. Li, X. Liu, N. R. Tallent, and K. J. Barker, “Evaluating modern gpu interconnect: Pcie, nvlink, nv-sli, nvswitch and gpudirect,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 1, pp. 94–110, Jan 2020.
- [4] Y. Huang, S. Di, X. Yu, G. Li, and F. Cappello, “cuszp: An ultra-fast gpu error-bounded lossy compression framework with optimized end-to-end performance,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–13.
- [5] S. A. RODRIGUEZ LUZARDO and P. Cardiff, “A general approach for running python codes in openfoam using an embedded pybind11 python interpreter,” *OpenFOAM® Journal*, vol. 2, p. 166–182, Dec. 2022. [Online]. Available: <http://dx.doi.org/10.51560/ofj.v2.79>
- [6] J. Hunt, *Monkey Patching*. Cham: Springer International Publishing, 2023, pp. 487–490. [Online]. Available: https://doi.org/10.1007/978-3-031-35122-8_43
- [7] Y. Zhang, Y. Dong, and K. Kawaguchi, “Investigating layer importance in large language models,” in *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, Y. Belinkov, N. Kim, J. Jumelet, H. Mohebbi, A. Mueller, and H. Chen, Eds. Miami, Florida, US: Association for Computational Linguistics, Nov. 2024, pp. 469–479. [Online]. Available: <https://aclanthology.org/2024.blackboxnlp-1.29/>
- [8] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” 2019.
- [9] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [10] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang,

- and L. Zettlemoyer, “Opt: Open pre-trained transformer language models,” 2022. [Online]. Available: <https://arxiv.org/abs/2205.01068>
- [11] S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff *et al.*, “Pythia: A suite for analyzing large language models across training and scaling,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 2397–2430.
- [12] P. Zhang, G. Zeng, T. Wang, and W. Lu, “Tinyllama: An open-source small language model,” 2024.
- [13] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, “Zero-infinity: breaking the gpu memory wall for extreme scale deep learning,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: <https://doi.org/10.1145/3458817.3476205>
- [14] K. Sayood, *Introduction to data compression / Khalid Sayood.*, fifth edition. ed., ser. The Morgan Kaufmann Series in Multimedia Information and Systems Series. Cambridge, MA: Elsevier Inc., 2018.
- [15] T. Liu, J. Wang, Q. Liu, S. Alibhai, T. Lu, and X. He, “High-ratio lossy compression: Exploring the autoencoder to compress scientific data,” *IEEE Transactions on Big Data*, vol. 9, no. 1, pp. 22–36, Feb 2023.
- [16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf
- [17] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. [Online]. Available: <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [18] M. Xu, D. Cai, W. Yin, S. Wang, X. Jin, and X. Liu, “Resource-efficient algorithms and systems of foundation models: A survey,” *ACM Comput. Surv.*, vol. 57, no. 5, Jan. 2025. [Online]. Available: <https://doi.org/10.1145/3706418>

- [19] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H₂O: Heavy-hitter oracle for efficient generative inference of large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.14048>
- [20] S. Li and T. Hoefler, “Chimera: efficiently training large-scale neural networks with bidirectional pipelines,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: <https://doi.org/10.1145/3458817.3476145>